

# Système de blockchain

## Fonctions Auxiliaires

Comme proposé en commentaire dans le fichier **BaseModule.oz**, le projet a été implémenté avec des fonctions auxiliaires dans le but de les réutiliser dans les fonctions **Decode** et **ExecuteBlockchain** et simplifier ces fonctions en réservant spécifiquement une tâche par fonction auxiliaire.

- Fonction Auxiliaire NumberToLetter

Cette fonction prend en entrée un nombre et le convertit en lettre selon le tableau donné.

- Fonction Auxiliaire DecodeHash

Cette fonction prend en entrée un hash et le convertit en string en réutilisant **NumberToLetter**.

- Fonction Auxiliaire CalculEffort

Cette fonction prend en entrée une valeur et calcule l'effort nécessaire pour effectuer une transaction.

- Fonction Auxiliaire CalculTransactionHash

Cette fonction prend en entrée un record Transaction calcule le hash de cette transaction.

- Fonction Auxiliaire CalculBlockHash

Cette fonction prend en entrée un record Block et calcule le hash d'un bloc.

- Fonction Auxiliaire IsValidTransaction

Cette fonction prend en entrée un record Transaction et State et vérifie si la transaction est valide selon les conditions données dans les consignes.

- Fonction Auxiliaire GetPreviousBlock

Cette fonction prend en entrée deux records CurrentBlock et Blockchain et retourne le bloc précédent.

- Fonction Auxiliaire IsValidBlock

Cette fonction prend en entrée Block, Blockchain et State et vérifie si un bloc est valide.

## Fonction Decode

Cette fonction prend en entrée une liste Blockchain, parcourt récursivement les records qui compose la liste et réutilise la fonction auxiliaire **DecodeHash** sur la liste en prenant en paramètre le hash du bloc concerné. Pour finalement concaténer avec le reste de la liste après avoir appliqué la fonction auxiliaire **DecodeHash** sur la suite des blocs.

## Fonction ExecuteBlockchain

Cette fonction se fait en quatre étapes. Premièrement, elle construit le state initial en utilisant une fonction auxiliaire **BuildState** qui transforme le record pris en entrée en un autre record user composé de balance et de nonce. Deuxièmement, elle ajoute l'effort aux transactions via une fonction auxiliaire **AddEffortTransactions** qui prend en paramètre une liste de transactions pour la parcourir et calculer l'effort des values en réutilisant la fonction auxiliaire **CalculEffort** et reconstruire les records des transactions. Troisièmement, elle met à jour le state de l'envoyeur mais également celui du récepteur via la fonction auxiliaire **UpdateState**. Enfin, la fonction auxiliaire **Loop** parcourt récursivement les transactions et filtre les transactions valides à l'aide de la fonction auxiliaire définie précédemment **IsValidTransaction** pour ensuite construire la blockchain et stocker dans des variables les résultats.

## État final de la blockchain

### ● Solde de chaque utilisateur et son dernier nonce

Voici l'output obtenu après compilation.

```
state(  
11:user(balance:97415 nonce:2)  
14:user(balance:70294080 nonce:8)  
15:user(balance:94650918 nonce:6)  
32:user(balance:5141804 nonce:7)  
37:user(balance:1239534151 nonce:0)  
49:user(balance:463119 nonce:1)  
60:user(balance:36160 nonce:3)  
61:user(balance:221043 nonce:4)  
66:user(balance:480590 nonce:2)  
73:user(balance:58497 nonce:4)  
86:user(balance:918173 nonce:2)  
87:user(balance:210757 nonce:1)  
100:user(balance:579432 nonce:3)  
)
```

### ● Blockchain finale

| Bloc | Hash   | PreviousHash | Hashes des transactions             |
|------|--------|--------------|-------------------------------------|
| 0    | 256424 | 0            | {22472, 38896, 81865, 61514, 51677} |
| 1    | 153662 | 256424       | {363117, 465569, 68551}             |
| 2    | 146614 | 153662       | {751304, 50438, 62480, 128728}      |
| 3    | 647331 | 146614       | {291119, 170096, 24257, ...}        |
| 4    | 102336 | 647331       | {247790, 139871, ...}               |
| 5    | 272434 | 102336       | {92750, ...}                        |
| 6    | 551027 | 272434       | {...}                               |
| 7    | 297173 | 551027       | {...}                               |

### ● Phrase secrète obtenue

"prof peter van royiarty is behind oz"

ce qui donne en français :

"Prof Peter Van Royiarty est derrière Oz."

## Extension 1 - Denylist

### ● Implémentation de l'extension Denylist

Pour implémenter l'extension Denylist, le squelette de **BaseModule.oz** a été réutilisé. Afin d'implémenter le système de liste noire, la fonction auxiliaire **IsValidTransaction** prend désormais un paramètre en plus qui est DenyList, soit la liste noire. La fonction vérifie si l'envoyeur (Sender) n'est pas dans la liste noire (DenyList) afin de déterminer si la transaction est valide ou non. Ensuite dans la fonction **ExecuteBlockchain**, une fonction auxiliaire **IncrementCount** a été implémentée, son objectif étant de compter les transactions par utilisateur afin d'ajouter, selon les spécifications données, les utilisateurs qui dépassent les trois transactions au sein d'un même bloc. La fonction auxiliaire **Loop** qui parcourt récursivement les transactions et filtre les transactions valides à l'aide de la fonction auxiliaire définie précédemment **IsValidTransaction** pour ensuite construire la blockchain et stocker dans des variables les résultats, se voit rajouter les paramètres Denylist et Counts qui permettent de gérer dynamiquement la liste noire.

### ● Solde de chaque utilisateur et son dernier nonce

```
state(  
11:user(balance:97415 nonce:2)  
14:user(balance:70226791 nonce:8)  
15:user(balance:94043702 nonce:6)  
32:user(balance:6107292 nonce:3)  
37:user(balance:1239534151 nonce:0)  
49:user(balance:463119 nonce:1)  
60:user(balance:36160 nonce:3)  
61:user(balance:221043 nonce:4)  
66:user(balance:457435 nonce:2)  
73:user(balance:81652 nonce:3)  
86:user(balance:918173 nonce:2)  
87:user(balance:210757 nonce:1)  
100:user(balance:288449 nonce:3)  
)
```

### ● Blockchain finale

| Bloc | Hash Bloc | Previous Hash | Hashes des transactions           |
|------|-----------|---------------|-----------------------------------|
| 0    | 256424    | 0             | 22472, 38896, 81865, 61514, 51677 |
| 1    | 153662    | 256424        | 363117, 465569, 68551             |
| 2    | 146614    | 153662        | 751304, 50438, 62480, 128728      |
| 3    | 356212    | 146614        | 170096, 24257, 15242              |
| 4    | 743877    | 356212        | 247790, 139871                    |
| 5    | 913975    | 743877        | 92750, ...                        |
| 6    | 168063    | 913975        | ...                               |
| 7    | 914209    | 168063        | ...                               |

### ● Phrase secrète obtenue

"prof petezpc h g qh gp ejzjwx xi"  
Qui n'a absolument aucun sens.

## Extension 2 - L'effort n'est pas gratuit

### ● Solde de chaque utilisateur et son dernier nonce

Pour implémenter cette extension, nous avons repris notre code **BaseModule.oz**. Il s'agissait d'une excellente base de travail. La majorité des fonctionnalités et des conditions étaient, effectivement, pour la plupart déjà implémentées. Il nous restait plus qu'à modifier les fonctions **IsValidTransaction** et **UpdateState**. La première gérant la validité d'une transaction, nous avons rajouté la ligne suivante :

$$UserInfo.balance < (Transaction.value + Transaction.effort)$$

Cette condition garantit que l'envoyeur peut couvrir les frais de transaction. Pour ce qui est de la deuxième fonction, il suffisait plus qu'à faire en sorte que l'effort de la transaction soit soustrait de la balance de l'envoyeur :

$$user(balance : State.(Sender).balance - Value - Effort \quad nonce : Transaction.nonce)$$

### ● Solde de chaque utilisateur et son dernier nonce

Voici l'output obtenu après compilation.

```
state(  
11:user(balance:97353 nonce:2)  
14:user(balance:70293640 nonce:8)  
15:user(balance:94650668 nonce:6)  
32:user(balance:5141523 nonce:7)  
37:user(balance:1239534151 nonce:0)  
49:user(balance:463088 nonce:1)  
60:user(balance:36035 nonce:3)  
61:user(balance:220919 nonce:4)  
66:user(balance:480496 nonce:2)  
73:user(balance:58341 nonce:4)  
86:user(balance:918127 nonce:2)  
87:user(balance:210726 nonce:1)  
100:user(balance:579275 nonce:3)  
)
```

### ● Blockchain finale

| Bloc | Hash   | PreviousHash | Hashes des transactions             |
|------|--------|--------------|-------------------------------------|
| 0    | 256424 | 0            | {22472, 38896, 81865, 61514, 51677} |
| 1    | 153662 | 256424       | {363117, 465569, 68551}             |
| 2    | 146614 | 153662       | {751304, 50438, 62480, 128728}      |
| 3    | 647331 | 146614       | {291119, 170096, 24257, ...}        |
| 4    | 102336 | 647331       | {247790, 139871, ...}               |
| 5    | 272434 | 102336       | {92750, ...}                        |
| 6    | 551027 | 272434       | {...}                               |
| 7    | 297173 | 551027       | {...}                               |

### ● Phrase secrète obtenue

La phrase secrète a changé et est la suivante : "prof peter van royiarty is behind oz".